SPM-Unit-2

2.1 Software Engineering Practices and its importance, Core principles.
2.2 Communication Practices, Planning Practices, Modelling practices, construction practices, software deployment (Statement and meaning of each principle for each practice).
2.3 Requirement Engineering:
Requirement Gathering and Analysis, Types of Requirements (Functional, Product, organizational, External Requirements), Eliciting Requirements, Developing Usecases, Building requirement models, Requirement Negotiation, Validation.
2.4 Software Requirement Specification: Need of SRS, Format, and its Characteristics.

2.1 Software Engineering Practices and its importance, Core principles.

Software Engineering is the systems engineering approach for software product/application development. It is an engineering branch associated with analyzing user requirements, design, development, testing, and maintenance of software products.

Core principles.

There are several basic principles of good software engineering approach

1. **Modularity**: Breaking down the software into smaller, independent, and reusable components or modules. This makes the software easier to understand, test, and maintain.
2. **Abstraction**: Hiding the implementation details of a module or component and exposing only the necessary information. This makes the software more flexible and easier to change.
3. **Encapsulation**: Wrapping the data and functions of a module or component into a single unit, and providing controlled access to that unit. This helps to protect the data and functions from unauthorized access and modification.

4. **DRY principle (Don't Repeat Yourself):** Avoiding duplication of code and data in the software. This makes the software more maintainable and less error-prone.
5. **KISS principle (Keep It Simple, Stupid)**: Keeping the software design and implementation as simple as possible. This makes the software more understandable, testable, and maintainable.
6. **YAGNI (You Ain't Gonna Need It):** Avoiding adding unnecessary features or functionality to the software. This helps to keep the software focused on the essential requirements and makes it more maintainable.
7. **SOLID principles:** A set of principles that guide the design of software to make it more maintainable, reusable, and extensible. This includes the Single Responsibility Principle, Open/Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, and Dependency Inversion Principle.
8. **Test-driven development:** Writing automated tests before writing the code, and ensuring that the code passes all tests before it is considered complete. This helps to ensure that the software meets the requirements and specifications.

   By following these principles, software engineers can develop software that is more reliable, maintainable, and extensible.
   It's also important to note that these principles are not mutually exclusive, and often work together to improve the overall quality of the software.

Software Engineering importance

Importance of Software Engineering

The importance of software engineering lies in the fact that a specific piece of Software is required in almost every industry, every business, and purpose. As time goes on, it becomes more important for the following reasons.

1. Reduces Complexity

Dealing with big Software is very complicated and challenging. Thus, to reduce the complications of projects, software engineering has great solutions. It simplifies complex problems and solves those issues one by one.

2. Handling Big Projects

Big projects need lots of patience, planning, and management, which you never get from any company. The company will invest its resources; therefore, it should be completed within the deadline. It is only possible if the company uses software engineering to deal with big projects without problems.

3. To Minimize Software Costs

Software engineers are paid highly as Software needs a lot of hard work and workforce development. These are developed with the help of a large number of codes. But programmers in software engineering project all things and reduce the things which are not needed. As a result of the production of Software, costs become less and more affordable for Software that does not use this method.

4. To Decrease Time

If things are not made according to the procedures, it becomes a huge loss of time. Accordingly, complex Software must run much code to get definitive running code. So, it takes lots of time if not handled properly. And if you follow the prescribed software engineering methods, it will save your precious time by decreasing it.

5. Effectiveness

Making standards decides the effectiveness of things. Therefore, a company always targets the software standard to make it more effective. And Software becomes more effective only with the help of software engineering.

6. Reliable Software

The Software will be reliable if software engineering, testing, and maintenance are given. As a software developer, you must ensure that the Software is secure and will work for the period or subscription you have agreed upon.

Why do we Need Software Engineering?

Most people don't give a second thought to new technologies as they make their life easier and more comfortable to drive. We need software engineering because software engineering is important in daily life. We have technology like Alexa only because we have software engineering. It has made things possible which are always beyond our imagination. Let's explore some points to answer why we need software engineering:

1. The rise of technology

2. Adding structure

3. Preventing issues

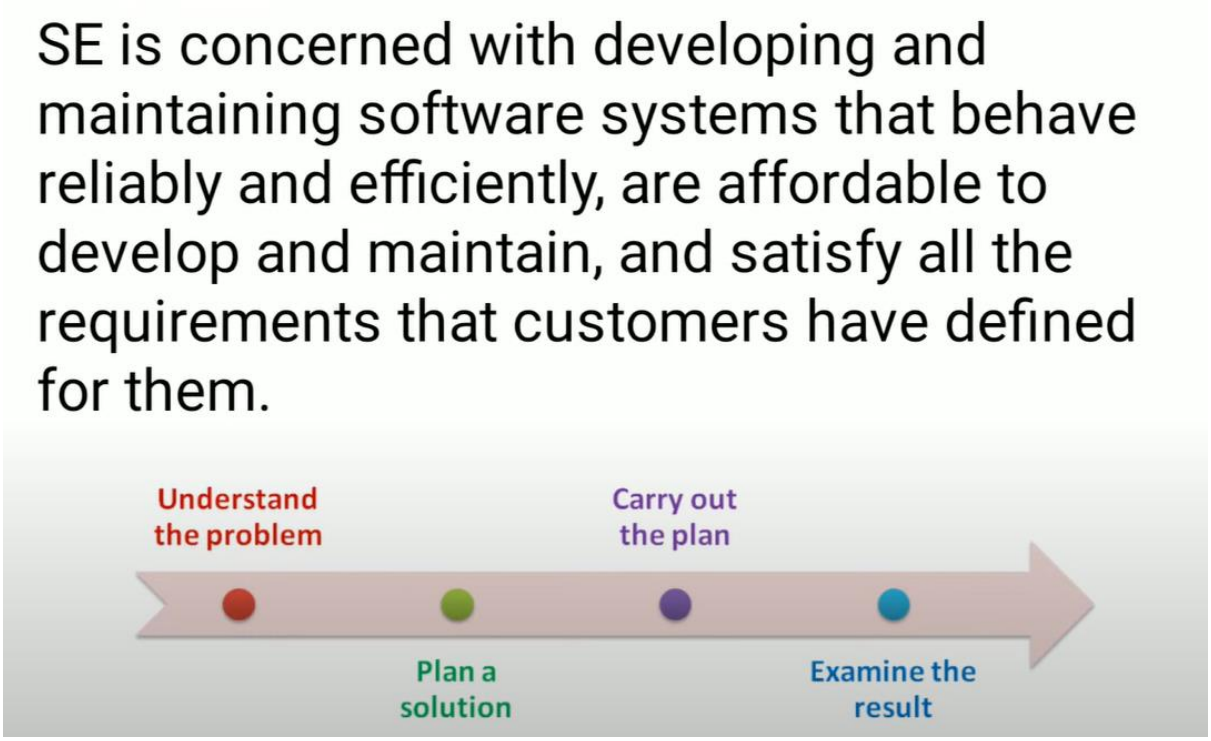
4. Huge Programming

5. Automation & AI

6. Research

Software Engineering Practices

The Essence of Software Engineering Practices:



Understand the problem (communication and analysis)

- ✔ *Who are the stakeholders?*
- ✔ *What are the unknowns?*
- ✔ *Can the problem be compartmentalized?*
- ✔ *Can the problem be represented graphically?*
- ✔ *Can an analysis model be created?*

Plan the solution: (modeling and software design).

- ✔ *Have you seen a similar problem before?*
- ✔ *Has a similar problem been solved? If so, is the solution reusable?*
- ✔ *Can sub-problems be defined?*
- ✔ *Can you represent a solution in a manner that leads to effective implementation?*

Carry out the plan: (code generation).

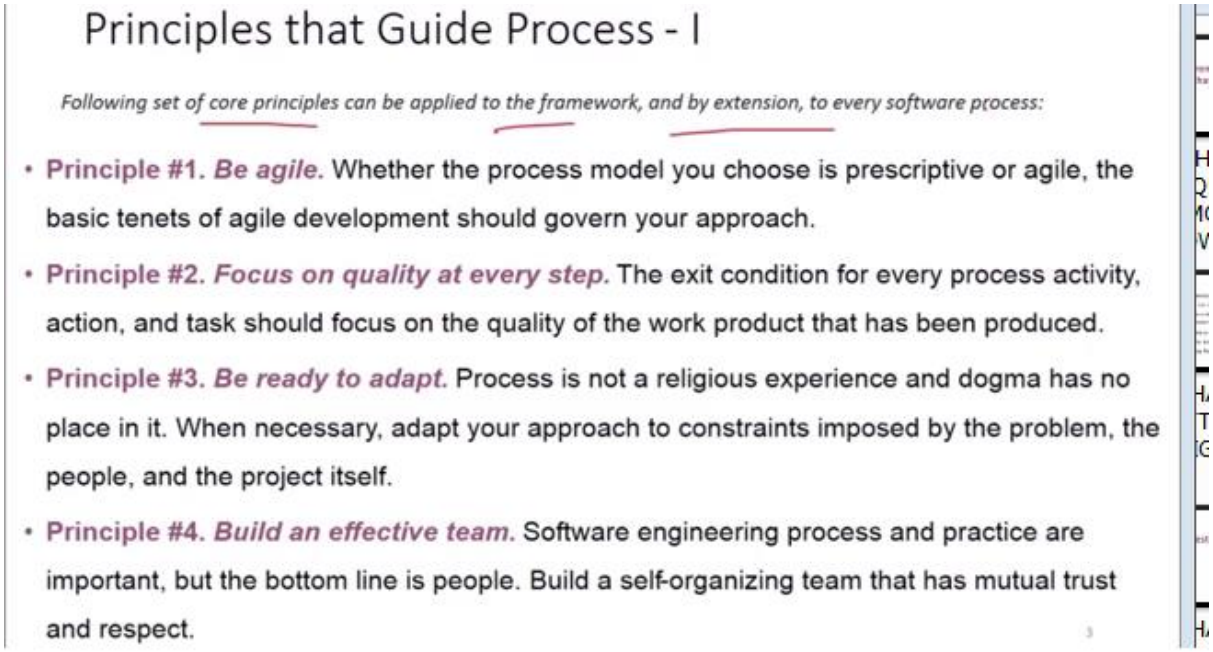
Does the solution conform to the plan?
Is each component part of the solution probably correct?

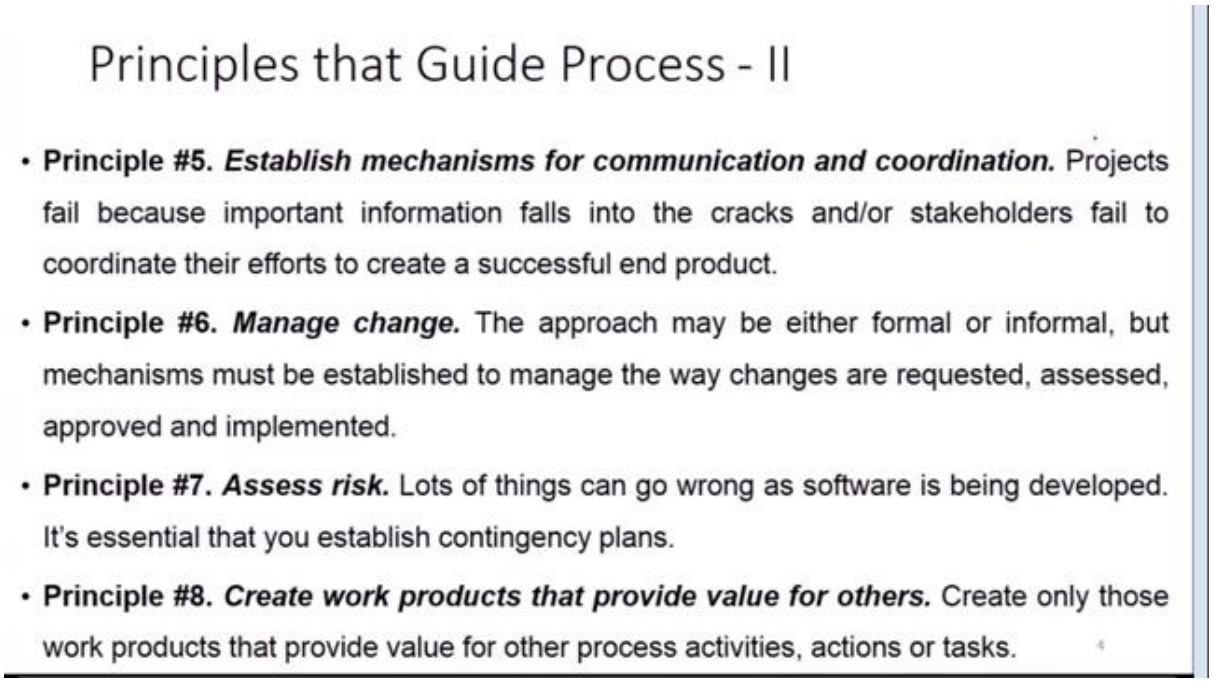
Examine the result: (testing and quality assurance).

Is it possible to test each component part of the solution?
Does the solution produce results that conform to the data, functions, features, and behavior that are required?

Core Principles that Guide i) Processes ii) Practices

Core Principles that Guide i) Processes:





Core Principles that Guide ii) Practices

Principles that Guide Practice - I

Software engineering practice has a single overriding goal—to deliver on-time, high- quality, operational software that contains functions and features that meet the needs of all stakeholders.

- **Principle #1.** ***Divide and conquer.*** Stated in a more technical manner, analysis and design should always emphasize *separation of concerns* (SoC).
- **Principle #2.** ***Understand the use of abstraction.*** At it core, an abstraction is a simplification of some complex element of a system used to communication meaning in a single phrase.
- **Principle #3. Strive for consistency.** A familiar context makes software easier to use.
- **Principle #4.** ***Focus on the transfer of information.*** Pay special attention to the analysis, design, construction, and testing of interfaces.

5

Principle #5. ***Build software that exhibits effective modularity.*** Separation of concerns (Principle #1) establishes a philosophy for software. *Modularity* provides a mechanism for realizing the philosophy.

Principle #6. ***Look for patterns.*** Brad Appleton [App00] suggests that: "The goal of patterns within the software community is to create a body of literature to help software developers resolve recurring problems encountered throughout all of software development.

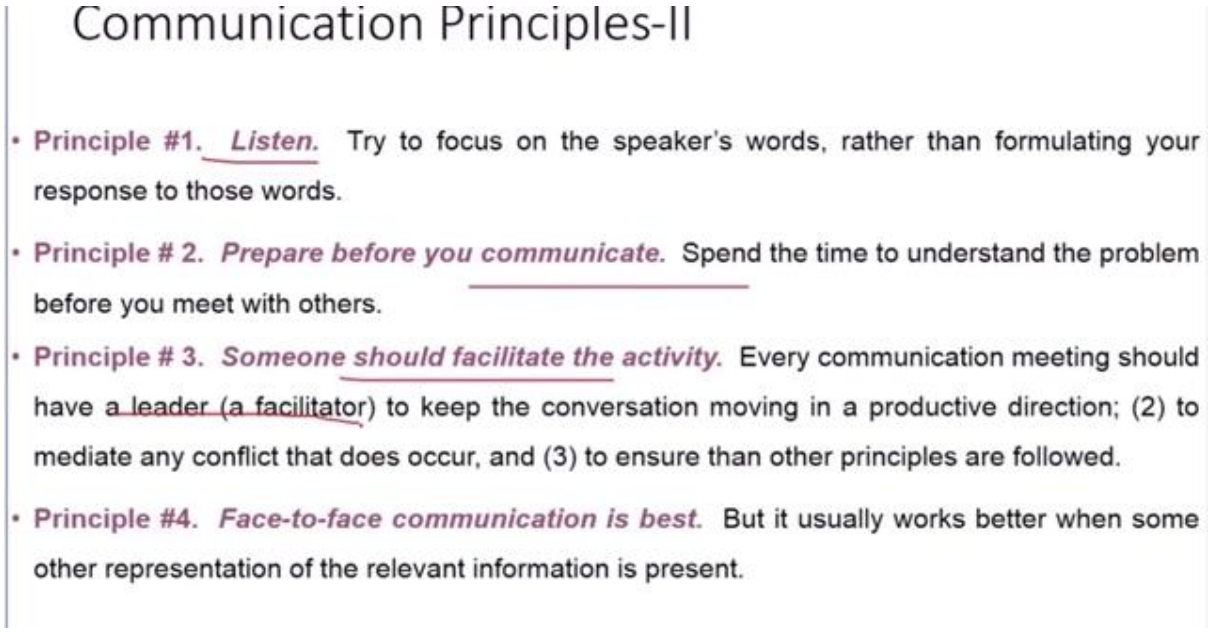
Principle #7. ***When possible, represent the problem and its solution from a number of different perspectives.***

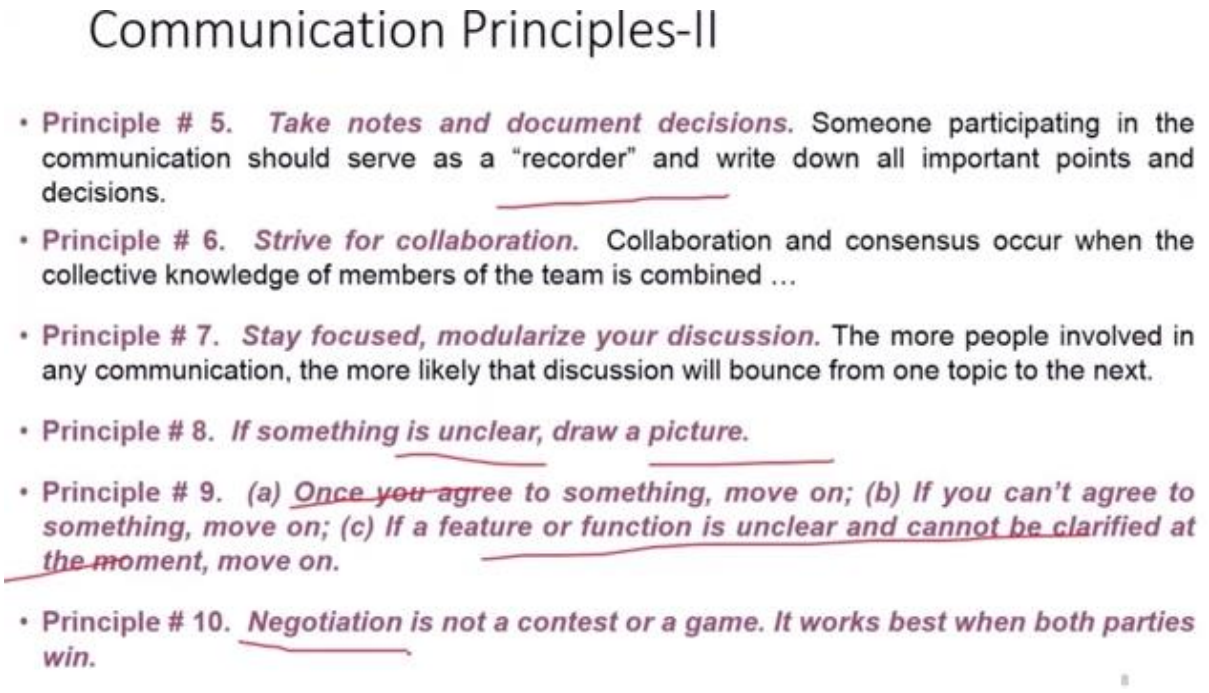
Principle #8. ***Remember that someone will maintain the software.***

6

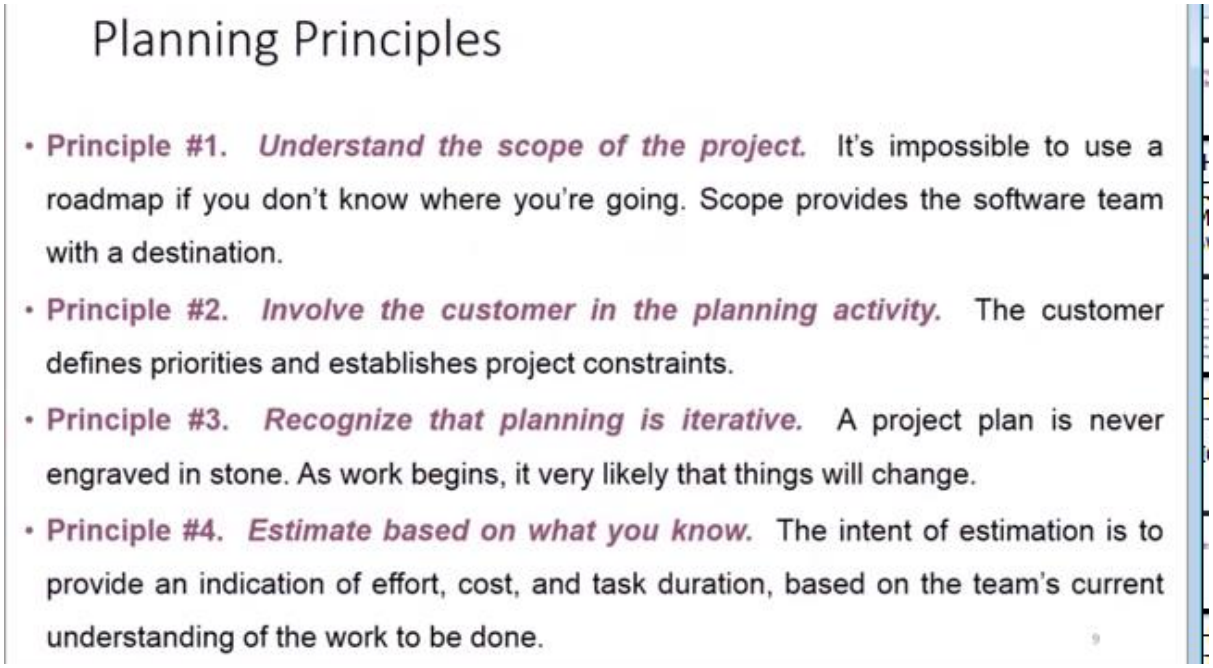
2.2 Communication Practices, Planning Practices, Modelling practices, construction practices, software deployment (Statement and meaning of each principle for each practice).

1) Communication Practices (Statement and meaning of each principle for each practice):





2) Planning Practices (Statement and meaning of each principle for each practice):



Planning Principles

- **Principle #1.** ***Understand the scope of the project.*** It's impossible to use a roadmap if you don't know where you're going. Scope provides the software team with a destination.
- **Principle #2.** ***Involve the customer in the planning activity.*** The customer defines priorities and establishes project constraints.
- **Principle #3.** ***Recognize that planning is iterative.*** A project plan is never engraved in stone. As work begins, it very likely that things will change.
- **Principle #4.** ***Estimate based on what you know.*** The intent of estimation is to provide an indication of effort, cost, and task duration, based on the team's current understanding of the work to be done.

9

Planning Principles

- **Principle #5.** ***Consider risk as you define the plan.*** If you have identified risks that have high impact and high probability, contingency planning is necessary.
- **Principle #6.** ***Be realistic.*** People don't work 100 percent of every day.
- **Principle #7.** ***Adjust granularity as you define the plan.*** *Granularity* refers to the level of detail that is introduced as a project plan is developed.
- **Principle #8.** ***Define how you intend to ensure quality.*** The plan should identify how the software team intends to ensure quality.
- **Principle #9.** ***Describe how you intend to accommodate change.*** Even the best planning can be obviated by uncontrolled change.
- **Principle #10.** ***Track the plan frequently and make adjustments as required.*** Software projects fall behind schedule one day at a time.

10

3) Modelling practices (Statement and meaning of each principle for each practice):

Modeling Principles

- In software engineering work, two classes of models can be created:
  - *Requirements models* (also called *analysis models*) represent the customer requirements by depicting the software in three different domains: the information domain, the functional domain, and the behavioral domain.
  - *Design models* represent characteristics of the software that help practitioners to construct it effectively: the architecture, the user interface, and component-level detail.

11

4) construction practices (Statement and meaning of each principle for each practice):

Construction Principles

- The construction activity encompasses a set of coding and testing tasks that lead to operational software that is ready for delivery to the customer or end-user.
- Coding principles and concepts are closely aligned programming style, programming languages, and programming methods.
- Testing principles and concepts lead to the design of tests that systematically uncover different classes of errors and to do so with a minimum amount of time and effort.

17

5) software deployment (Statement and meaning of each principle for each practice):

Deployment Principles

- **Principle #1.** ***Customer expectations for the software must be managed.*** Too often, the customer expects more than the team has promised to deliver, and disappointment occurs immediately.
- **Principle #2.** ***A complete delivery package should be assembled and tested.***
- **Principle #3.** ***A support regime must be established before the software is delivered.*** An end-user expects responsiveness and accurate information when a question or problem arises.
- **Principle #4.** ***Appropriate instructional materials must be provided to end-users.***
- **Principle #5.** ***Buggy software should be fixed first, delivered later.***

2.3 Requirement Engineering:
Requirement Gathering and Analysis, Types of Requirements (Functional, Product, organizational, External Requirements), Eliciting Requirements, Developing Usecases, Building requirement models, Requirement Negotiation, Validation.

2.3 Requirement Engineering:

The process to gather the software requirements from client, analyze and document them is known as requirement engineering.

The goal of requirement engineering is to develop and maintain sophisticated and descriptive '**System Requirements Specification' document.**

According to IEEE standard 729, a requirement is defined as follows:

- A condition or capability needed by a user to solve a problem or achieve an objective
- A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification or other formally imposed documents
- A documented representation of a condition or capability as in 1 and 2.

Requirement Gathering and Analysis

- **Interface traceability table** : Shows how requirements relate to both internal and external system interfaces.

- Traceability is useful in case of large and complex system to determine the connections between the requirements so as to understand how a change in one requirement affects the other requirement and how it will affect the different aspects of the system to be built.

2.6 Requirement Gathering and Analysis

- It is about collecting the requirements, needs and constraints.
- This task collects the information about :
  - Domain - problems and laws
  - Existing standards and systems
  - Existing specifications
- The Q and A session does not lead to a very detail elicitation of requirements and so the following approaches are used for successfully eliciting (gathering) the requirements.

2.6.1 Collaborative Requirements Gathering

- The Q and A session during inception only establishes the scope of the problem and overall perception of a solution. Out of these initial meetings, the stakeholders write a one or two page "product request". A meeting place, time, and date are selected and a facilitator is chosen.
- The software team and the other stakeholders are invited to attend this meeting. The product request is distributed to all attendees before the meeting date. Before coming to the meeting, each attendee is asked to make a list of objects that are part of environment that surrounds the system, that are to be produced by the system and objects that are used by the system to perform its functions. Also, he is asked to list the services or functions that act on these objects.
- Finally, list of constraints (like business rules, cost and size) and performance criteria (i.e. speed, accuracy) are also developed.
- Then comes the actual meeting (collaborative requirement gathering) date. In this gathering, a team of stakeholders and developers work together to identify the problem, propose elements of solution, negotiate different approaches and specify a preliminary set of solution requirements.
- A collaborative requirement gathering is about :
  - Conducting meetings which are attended by both software engineers and customers.
  - Rules for preparation and participation are established.
  - An agenda is suggested to cover all important points and encourage the free flow of ideas.
  - A 'facilitator' (may be a customer or developer) controls the meeting.
  - A 'definition mechanism'(can be worksheets, flip charts, wall stickers or an electronic bulletin board, chat room or virtual forum) is used.
- The goal of such gathering is :
  - To identify the problem
  - Propose elements of the solution
  - Negotiate different approaches Specify a preliminary set of solution requirements.
- **Example :** Consider the "Safe Home" project.
  - The list of *objects* involved in this project are control panel, smoke detectors, window and door sensors, motion detectors, an alarm, an event (sensor has been activated), a display, a PC, telephone numbers, a telephone call and so on.
  - The list of *services* might include configuring the system, setting the alarm, monitoring the sensors, dialing the phone, programming the control panel and reading the display. All these services act on objects.
  - The list of *constraints* might be as - the system must recognize when sensors are not operating, must be user-friendly, must interface directly to a standard phone line.

- The *performance criteria* might be as - a sensor event should be recognized within a second, an event priority scheme should be implemented.

- As the *gathering begins*, first topic of discussion is the need and justification for the new product - everyone should agree that the product is justified. Once the agreement has been established, each participant presents his list for discussion. The lists can be pinned to the walls of the room and they can also be posted on an electronic bulletin board or in a chat room environment. Each listed entry should be capable of being manipulated separately (deleted, added or modified). At this stage, critique and debate are strictly prohibited.

- Then, a combined list is created by the group so as to eliminate the redundant entries and adds any new ideas that come up during the discussion. Next, the facilitator coordinates the discussion to shorten, lengthen or reword the combined list so as to properly reflect the system to be developed.

- Now, the team is divided into smaller sub teams; each works to develop mini specifications for one or more entries of the list. Each mini specification is the elaboration of the word included in the list.

- For example, mini specification of the control panel is - It is a wall mounted unit and is 9*5 inches in size, has wireless connectivity to sensors and a PC, user interaction occurs through a keyboard having 12 keys, a 2*2 inch CD display provides the user feedback, provides interactive prompts, echo and similar functions.

- After the mini-specs are completed, each participant makes a list of validation criteria for the system. All these lists are combined and a consensus list of validation criteria is created. Finally, one or more participants are assigned the task of writing a complete draft specification using all inputs from the meeting.

2.6.2 Quality Function Deployment

- It would be very helpful to have a technique that can assist in accurate requirement gathering. Quality Function Deployment (QFD) is a technique that enables a software team to specify clearly the customer's wants and needs, and then evaluate each proposed object or service capability in terms of its impact on meeting those needs.

- QFD is a technique that translates customer requirements into technical requirements for software. QFD focuses on customer needs throughout the software engineering process.

- QFD identifies 3 types of requirements :

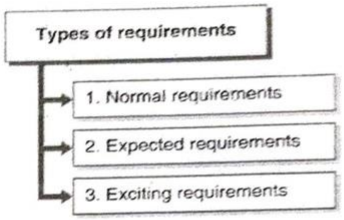


Fig. 2.6.1 : Types of requirements

1. Normal requirements

It reflects objectives and goals stated for a system during meetings with the customer. If these requirements are present, the customer is satisfied.

Example : Graphical displays, specific system functions and defined levels of performance.

2. Expected requirements

These are implicit to the system and may be so fundamental that the customer does not explicitly state them. Absence of such requirements causes customer dissatisfaction.

Example : Ease of human-machine interaction, overall operational correctness and reliability and ease of software installation.

3. Exciting requirements

- These reflect features that go beyond the customer's expectations and prove to be very satisfying when present.

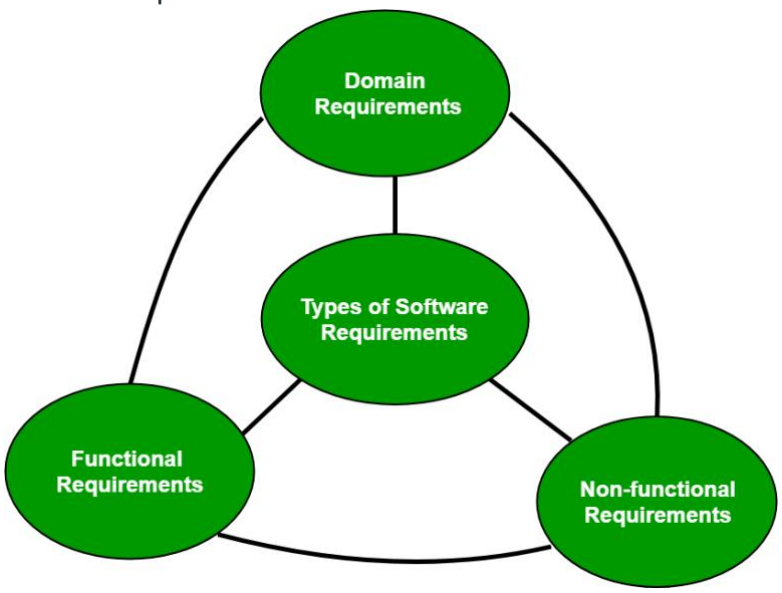
  Example : Customer may request a Word processing software with standard features but he is pleased when he sees that the product delivered to him has number of page layout capabilities along with basic features.

- In meetings with the customer, function deployment determines the value of each function that is required for the system. Information deployment identifies data objects and events that the system must consume and

Types of Requirements:

Main types of software requirement can be of 3 types:

1. Functional requirements
2. Non-functional requirements
3. Domain requirements



1. Functional Requirements:

These are the requirements that the end user specifically demands as basic facilities that the system should offer. It can be a calculation, data manipulation, business process, user interaction, or any other specific functionality which defines what function a system is likely to perform.

All these functionalities need to be necessarily incorporated into the system as a part of the contract.

For example, in a hospital management system, a doctor should be able to retrieve the information of his patients.

Each high-level functional requirement may involve several interactions or dialogues between the system and the outside world.

2. Non-functional requirements:

These are the quality constraints that the system must satisfy according to the project contract.

Nonfunctional requirements, not related to the system functionality, rather define how the system should perform.

They basically deal with issues like:

- Portability
- Security
- Maintainability
- Reliability
- Scalability
- Performance
- Reusability
- Flexibility

Three classes of Non-Functional Requirements:

i. Product requirements

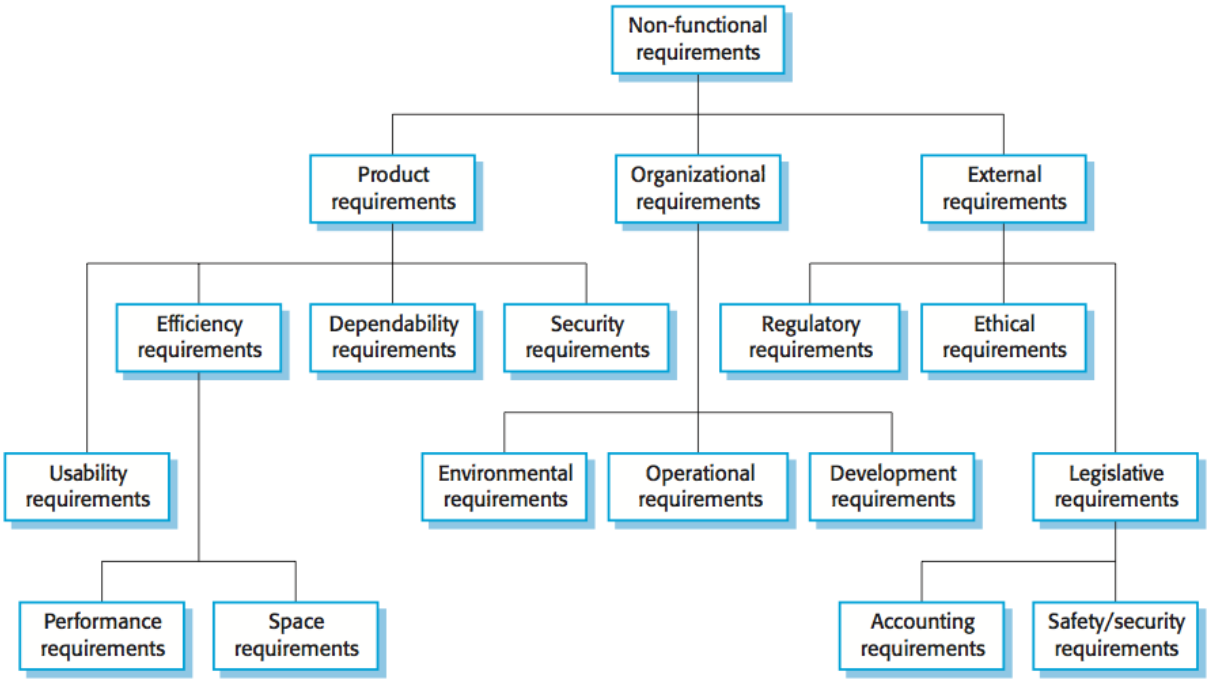
Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.

ii. Organizational requirements

Requirements which are a consequence of organizational policies and procedures e.g. process standards used, implementation requirements, etc.

iii. External requirements

Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.



The process of specifying non-functional requirements requires the knowledge of the functionality of the system, as well as the knowledge of the context within which the system will operate.

3. Domain requirements:

Domain requirements are the requirements which are characteristic of a particular category or domain of projects. Domain requirements engineering is a continuous process of proactively defining the requirements for all foreseeable applications to be developed in the software product line.

The **basic functions that a system of a specific domain must necessarily exhibit** come under this category.

For instance, in an academic software that maintains records of a school or college, the functionality of being able to access the list of faculty and list of students of each grade is a domain requirement.

These requirements are therefore identified from that domain model and are not user specific.

Other common classifications of software requirements can be:

1. User requirements:
2. System requirements:
3. Business requirements**:**
4. Regulatory requirements:
5. Interface requirements**:**
6. Design requirements:

By classifying software requirements, it becomes easier to manage, prioritize, and document them effectively. It also helps ensure that all important aspects of the system are considered during the development process.

Advantages of classifying software requirements include:

1. **Better organization:** Classifying software requirements helps organize them into groups that are easier to manage, prioritize, and track throughout the development process.
2. **Improved communication:** Clear classification of requirements makes it easier to communicate them to stakeholders, developers, and other team members. It also ensures that everyone is on the same page about what is required.
3. **Increased quality:** By classifying requirements, potential conflicts or gaps can be identified early in the development process. This reduces the risk of errors, omissions, or misunderstandings, leading to higher quality software.
4. **Improved traceability:** Classifying requirements helps establish traceability, which is essential for demonstrating compliance with regulatory or quality standards.

Disadvantages of classifying software requirements include:

1. **Complexity**: Classifying software requirements can be complex, especially if there are many stakeholders with different needs or requirements. It can also be time-consuming to identify and classify all the requirements.
2. **Rigid structure**: A rigid classification structure may limit the ability to accommodate changes or evolving needs during the

development process. It can also lead to a siloed approach that prevents the integration of new ideas or insights.

3. **Misclassification**: Misclassifying requirements can lead to errors or misunderstandings that can be costly to correct later in the development process.

Requirement Engineering:
Eliciting Requirements, Developing Usecases, Building requirement models, Requirement Negotiation, Requirement Validation.

Requirement Engineering:

The process to gather the software requirements from client, analyze and document them is known as requirement engineering.

The goal of requirement engineering is to develop and maintain sophisticated and descriptive 'System Requirements Specification' document'.

Requirement Engineering Process:

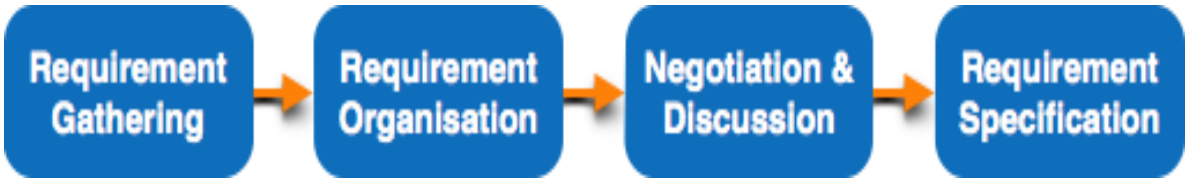
It is a four step process, which includes –

- Feasibility Study
- Requirement Gathering
- Software Requirement Specification
- Software Requirement Validation

Requirement Engineering:
Eliciting Requirements:

ELICITING REQUIREMENTS

Requirements elicitation (also called ***requirements gathering***) combines elements of problem solving, elaboration, negotiation, and specification



Collaborative Requirements Gathering

Many different approaches to collaborative requirements gathering have been proposed. Each makes use of a slightly different scenario, but all apply some variation on the following **basic guidelines:**

• Meetings are conducted and attended by both software engineers and other stakeholders.
• Rules for preparation and participation are established.
• An agenda is suggested that is formal enough to cover all important points but informal enough to encourage the free flow of ideas.
• A "facilitator" (can be a customer, a developer, or an outsider) controls the meeting.
• A "definition mechanism" (can be work sheets, flip charts, or wall stickers or
• an electronic bulletin board, chat room, or virtual forum) is used.

The goal is to identify the problem, propose elements of the solution, negotiate different approaches, and specify a preliminary set of solution requirements in an atmosphere that is conducive to the accomplishment of the goal.
During inception basic questions and answers establish the scope of the problem and the overall perception of a solution. Out of these initial meetings, the developer and customers write a **one- or two-page "product request**."

A meeting place, time, and date are selected; a facilitator is chosen; and attendees from the software team and other stakeholder organizations are invited to participate. The product request is distributed to all attendees before the meeting date.

While reviewing the product request in the days before the meeting, each attendee is asked to make a list of objects that are part of the environment that surrounds the system, other objects that are to be produced by the system, and objects that are used by the system to perform its functions.
In addition, each attendee is asked to make another list of services that manipulate or interact with the objects.

Finally, lists of constraints (e.g., cost, size, business rules) and performance criteria (e.g., speed, accuracy) are also developed. The attendees are informed that the lists are not expected to be exhaustive but are expected to reflect each person's perception of the system. 57 Software Engineering (R15)

The lists of objects can be pinned to the walls of the room using large sheets of paper, stuck to the walls using adhesive-backed sheets, or written on a wall board. After individual lists are presented in one topic area, the group creates a combined list by eliminating redundant entries, adding any new ideas that come up during the discussion, but not deleting anything.

Quality Function Deployment
Quality function deployment (QFD) is a quality management technique that translates the needs of the customer into technical requirements for software. QFD "**concentrates on maximizing customer satisfaction from the software engineering process**". To accomplish this, QFD emphasizes an understanding of what is valuable to the customer and then deploys these values throughout the engineering process.

QFD identifies **three** types of requirements :
• **Normal requirements.** The objectives and goals that are stated for a product or system during meetings with the customer. If these requirements are present, the customer is satisfied. Examples of normal requirements might be requested types of graphical displays, specific system functions, and defined levels of performance.
• **Expected requirements.** These requirements are implicit to the product or system and may be so fundamental that the customer does not explicitly state them. Their absence will be a cause for significant dissatisfaction.
• **Exciting requirements.** These features go beyond the customer's expectations and prove to be very satisfying when present.

Although QFD concepts can be applied across the entire software process, QFD uses customer interviews and observation, surveys, and examination of historical data as raw data for the requirements gathering activity. These data are then translated into a table of requirements— called the ***customer voice table***—that is reviewed with the customer and other stakeholders.

Usage Scenarios
As requirements are gathered, an overall vision of system functions and features begins to materialize.

However, it is difficult to move into more technical software engineering activities until you understand how these functions and features will be used by different classes of end users.

To accomplish this, developers and users can create a set of scenarios that identify a thread of usage for the system to be constructed. The scenarios, often called ***use cases***, provide a description of how the system will be used.

Elicitation Work Products
The work products produced as a consequence of requirements elicitation will vary depending on the size of the system or product to be built. For most systems, the work products include

• A statement of need and feasibility.
• A bounded statement of scope for the system or product.
• A list of customers, users, and other stakeholders who participated in requirements elicitation.
• A description of the system's technical environment.
• A list of requirements and the domain constraints that apply to each.
• A set of usage scenarios that provide insight into the use of the system or product under different operating conditions.
• Any prototypes developed to better define requirements.

Each of these work products is reviewed by all people who have participated in requirements elicitation.

Requirement Engineering:
Developing Usecases:

DEVELOPING USE CASES

Use cases are defined from an actor's point of view. An actor is a role that people (users) or devices play as they interact with the software.
The first step in writing a use case is to define the set of "actors" that will be involved in the story. *Actors* are the different people (or devices) that use the system or product within the context of the function and behavior that is to be described.

Actors represent the roles that people (or devices) play as the system operates. Defined somewhat more formally, an actor is anything that communicates with the system or product and that is external to the system itself.

Every actor has one or more goals when using the system. It is important to note that an actor and an end user are not necessarily the same thing.

A typical user may play a number of different roles when using a system, whereas an actor represents a class of external entities (often, but not always, people) that play just one role in the context of the use case. Different people may play the role of each actor.

Because requirements elicitation is an evolutionary activity, not all actors are identified during the first iteration. It is possible to identify **primary actors** during the first iteration and **secondary actors** as more is learned about the system.

Primary actors interact to achieve required system function and derive the intended benefit from the system.

Secondary actors support the system so that primary actors can do their work. Once actors have been identified, use cases can be developed.

Jacobson suggests a number of questions that should be answered by a use case:
• Who is the primary actor, the secondary actor(s)?
• What are the actor's goals?
• What preconditions should exist before the story begins?
• What main tasks or functions are performed by the actor?
• What exceptions might be considered as the story is described?
• What variations in the actor's interaction are possible?
• What system information will the actor acquire, produce, or change?
• Will the actor have to inform the system about changes in the external environment?
• What information does the actor desire from the system?
• Does the actor wish to be informed about unexpected changes?

The basic use case presents a high-level story that describes the interaction between the actor and the system.

Requirement Engineering:
Building requirement models

BUILDING THE REQUIREMENTS MODEL

The intent of the analysis model is to provide a description of the required informational, functional, and behavioral domains for a computer-based system.

The model changes dynamically as you learn more about the system to be built, and other stakeholders understand more about what they really require..

Elements of the Requirements Model

The specific elements of the requirements model are dictated by the analysis modeling method that is to be used. However, a set of generic elements is common to most requirements models.

• **Scenario-based elements.** The system is described from the user's point of view using a scenario-based approach.

• **Class-based elements.** Each usage scenario implies a set of objects that are manipulated as an actor interacts with the system. These objects are categorized into classes—a collection of things that have similar attributes and common behaviors.

• **Behavioral elements.** The behavior of a computer-based system can have a profound effect on the design that is chosen and the implementation approach that is applied. Therefore, the requirements model must provide modeling elements that depict behavior.

• **Flow-oriented elements.** Information is transformed as it flows through a computer- based system. The system accepts input in a variety of forms, applies functions to transform it, and produces output in a variety of forms.

Analysis Patterns

Analysis patterns suggest solutions (e.g., a class, a function, a behavior) within the application domain that can be reused when modeling many applications.

Geyer-Schulz and Hahsler suggest two benefits that can be associated with the use of analysis patterns:

1. **First**, analysis patterns speed up the development of abstract analysis models that capture the main requirements of the concrete problem by providing reusable analysis models with examples as well as a description of advantages and limitations.

2. **Second**, analysis patterns facilitate the transformation of the analysis model into a design model by suggesting design patterns and reliable solutions for common problems.

Analysis patterns are integrated into the analysis model by reference to the pattern name.

Requirement Engineering:
Requirement Negotiation

NEGOTIATING REQUIREMENTS

The intent of negotiation is to develop a project plan that meets stakeholder needs while at the same time reflecting the real-world constraints (e.g., time, people, budget) that have been placed on the software team. The best negotiations strive for a "**win-win**" result.

That is, stakeholders win by getting the system or product that satisfies the majority of their needs and you win by working to realistic and achievable budgets and deadlines.

Boehm defines a set of negotiation activities at the beginning of each software process iteration. Rather than a single customer communication activity, the following activities are defined:

1. Identification of the system or subsystem's key stakeholders.
2. Determination of the stakeholders' "win conditions."
3. Negotiation of the stakeholders' win conditions to reconcile them into a set of win-win conditions for all concerned.

Successful completion of these initial steps achieves a win-win result, which becomes the key criterion for proceeding to subsequent software engineering activities.

Requirement Engineering:
Validation.

VALIDATING REQUIREMENTS

As each element of the requirements model is created, it is examined for inconsistency, omissions, and ambiguity. The requirements represented by the model are prioritized by the stakeholders and grouped within requirements packages that will be implemented as software increments.

A review of the requirements model addresses the following questions:

• Is each requirement consistent with the overall objectives for the system/ product?

• Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?

• Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?

• Is each requirement bounded and unambiguous?

• Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?

• Do any requirements conflict with other requirements?

• Is each requirement achievable in the technical environment that will house the system or product?

• Is each requirement testable, once implemented?

• Does the requirements model properly reflect the information, function, and behavior of the system to be built?

• Has the requirements model been "partitioned" in a way that exposes progressively more detailed information about the system?

• Have requirements patterns been used to simplify the requirements model?

• Have all patterns been properly validated? Are all patterns consistent with customer requirements?

These and other questions should be asked and answered to ensure that the requirements model is an accurate reflection of stakeholder needs and that it provides a solid foundation for design.

2.4 Software Requirement Specification: Need of SRS, Format, and its Characteristics.

2.7 Software Requirement Specification (SRS)

Q. Explain the concept of software requirement specification. **(W-17, 4 Marks)**

- Specification is concerned with documenting the requirements and this document is maintained over the life of the project. This is the most fundamental condition for successful implementation of the project.
- Specification is not limited to just text document, it can include graphical notations, mathematical specifications, user scenarios or prototypes.
- Specification includes a set of use cases that describe how the user interacts with the software. Use cases are also known as functional requirements. In addition to use cases, specification also includes nonfunctional requirements such as performance requirements and quality standards.

- Specification is the complete description of the behavior of the system to be developed. It is the final work product produced by the requirements engineer, which serves as the foundation for subsequent software engineering activities.
- The basic issues that the SRS shall address are the following :

  i. **Functionality :** behaviour or services provided by the software.

  ii. **External interfaces :** external interactions of the software such as with end users, system's hardware or external software.

  iii. **Performance :** the throughput, the availability, the response time, the recovery time of the software.

  iv. **Quality attributes :** portability, correctness, maintainability, security, etc.

  v. **Constraints :** implementation language, policies for database integrity, resource limits, operating environments etc.

  Example : The ATM system is dependent on the network in the village areas. Due to flood and consequent network jams there will be a delay in the transactions.

2.7.1 Benefits (Need) of SRS

Q. What is SRS ? Explain importance of SRS. **(S-16, S- 17, 4 Marks)**

Q. State need of software requirement specification (SRS). **(S- 19, 2 Marks)**

- **SRS forms the basis for agreement between customers and development organization on what the software product is expected to do.** Complete descriptions of the functions that are expected from the software are specified in the SRS. This will help the end users to verify whether the software meets the specified needs or not and if not, then how the software must be manipulated so as to meet their requirements.

- **SRS reduces the development effort.** The work of preparing SRS forces various stakeholders i.e. various concerned groups in the customer's organization to think thoroughly of all their requirements before the project design begins, thus reducing the efforts needed for redesigning, recoding, and retesting. Careful inspection of the requirements specified in SRS can reveal the left-outs, misinterpretations and inconsistencies early in the development cycle; hence it becomes easier to correct these problems.
- **SRS forms a base for cost estimation and project scheduling.** As complete description of the product to be developed is specified in the SRS, it helps in estimating the project costs and can be used to obtain approval from the customer for the decided price estimates.
- **SRS provides a baseline for verification and validation.** Software development team can develop their verification and validation plans or say, test plans much more effectively from a well-prepared SRS document. As SRS provides a baseline against which requirement confirmation can be measured.
- **SRS facilitates transfer of new software to new environments.** SRS makes it easier to transfer the software product to new clients or new hardware machines. Therefore, developers feel easier to transfer the software to new clients and customers feel easier to transfer the software on other systems of their organization.
- **SRS serves as a basis for software improvement.** SRS describes the product features and not the process of project development; therefore, it serves as a basis for enhancements by allowing the developers to do any later modification if required on the finished product. But then, SRS needs to be altered in that case i.e. SRS forms a base for continuous product evaluation.

2.7.2 Characteristics of SRS

- ***Great SRS leads to Great Product :*** We have to keep in mind that the goal is to create great products and great software. A great product can be created only from a great specification. Systems and software these days are so complex that to get on with the design before knowing what you are going to build is foolish and risky.

- A great SRS has following quality factors :
  - **Correct** : A set of requirements is correct if and only if every requirement stated therein represents something required of the system to be built." Davis (1993).
  - **Unambiguous** : The reader of a requirement statement should be able to draw only one interpretation of it.
  - **Incorrect Example** : "The system should not accept password longer than 8 characters".
- The above stated requirement can have multiple interpretation as given below :
  - The system should not allow user to enter more than 8 characters in the password field.
  - The system should truncate the entered data to 8 characters.
  - The system should display an error message if user enters more than 8 characters in the password field.
- **Correct Example** : "The system should not accept passwords longer than 8 characters in the password field. If the user enters more than 8 characters while entering the password, an error message should prompt the user to correct the same."
- **Complete** : No requirements should be missing. A complete requirement leaves no room for guessing. Include all significant requirements, whether related to functionality, performance, design constraints, attributes, or external interfaces.
- **Consistent** : A consistent requirement does not conflict with other requirements in the requirement specification.
- **Incorrect Example** :

  REQ1 : "The birth date should be entered in mm/dd/yyyy format".

  REQ2 : "The birth date should be entered in dd/mm/yyyy format".
- **Correct Example** :

  REQ1 : "For the US users the birth date should be entered in mm/dd/yyyy format"

  REQ2 : "For the French users the birth date should be entered in mm/dd/yyyy format"

- **Ranked for Importance :** Requirements should be achievable. But sometimes, few requirements are not attainable
- **Verifiable** : Don't put in requirements like – "It should provide the user a fast response" or "The system should never crash". Instead, provide a quantitative requirement like: "Every key stroke should provide a user response within 100 milliseconds."
- **Traceable** : Each requirement should be expressed only once and should not overlap with another requirement. Every requirement has a unique identification number so that requirements can be easily traced throughout the specification.
- **Adaptability :** Can the requirement be changed without a large impact on other requirements ?

2.7.3 Components of SRS

Q. Explain general format of Software Requirement Specification (SRS). **(W-16, 4 Marks)**

Table 2.7.1 is the IEEE standards SRS format which depicts the components of a SRS document.

Table 2.7.1

1. INTRODUCTION
1.1 PRODUCT OVERVIEW (Product description)
1.2 PURPOSE (Product description)
1.3 SCOPE Write the benefits, objectives and goals.
1.4 AUDIENCE (lists the users of SRS such as developers, project managers, marketing staff, users, testers, and documentation writers.)
1.5 DEFINITIONS, ACRONYMS AND ABBREVIATIONS (defines all the terms necessary to properly interpret the SRS).
1.6 CONVENTIONS (describes the standard conventions followed while writing this SRS such as fonts or special significant highlights)
1.7 REFERENCES (lists the reference documents or web page address used as reference)

2. OVERALL DESCRIPTION

2.1 PRODUCT PERSPECTIVE

(lists the reference documents or web page address used as reference)

2.2 PRODUCT FUNCTIONALITY

(designs such as DFDs that describe major functions of the system)

2.3 USER CHARACTERISTICS

Identify various users who will be using this product. Differentiate them based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience.

2.4 OPERATING ENVIRONMENT

(Designs that describe that shows the major components of the overall system, subsystem interconnections, and external interface.)

2.5 DESIGN AND IMPLEMENTATION CONSTRAINTS

Describe the issues of the developers such as - hardware limitations, interfaces to other applications, specific technologies, tools, and databases to be used, language requirements, communications protocols, security considerations, design conventions.

2.6 USER DOCUMENTATION

List out the user manuals, on-line help, and tutorials that will be delivered along with the software.

2.7 ASSUMPTIONS AND DEPENDENCIES

List any assumed factors that could affect the requirements stated in the SRS. These include third-party components or issues around the development or operating environment. The project could be affected if these assumptions are incorrect, are not shared, or change.

Also list out the dependencies such as software components that you plan to reuse from another project.

3. SPECIFIC REQUIREMENTS

3.1 EXTERNAL INTERFACE REQUIREMENTS

3.1.1 USER INTERFACES

List out the different screens that will be available to the user.

3.1.2 HARDWARE INTERFACES

List out any special libraries that are used to communicate with your software mention them.

3.1.3 SOFTWARE INTERFACES

(Describes databases, operating systems and tools, services and the data that will be shared across software components)

3.1.4 COMMUNICATIONS REQUIREMENTS

(Describes the communication related requirements such as e-mail, web browser, network server communications protocols, electronic forms and the communication standards such as FTP or HTTP.)

3.2 FUNCTIONAL REQUIREMENTS

(describes the functional requirements in detail with proper explanations regarding each and every function.)

3.3 BEHAVIOUR REQUIREMENTS

3.3.1 USE CASE VIEW

Draw a use case diagram that will illustrate the entire system and all possible actors.

4. OTHER NON FUNCTIONAL REQUIREMENTS

4.1 RELIABILITY

4.2 AVAILABILITY

4.3 SAFETY and SECURITY

(describes the functional requirements in detail with proper explanations regarding each and every function.)

4.4 MAINTAINABILITY

4.5 PORTABILITY